HƯỚNG DẪN XÂY DỰNG ỨNG DỤNG GỌI VIDEO CALL GIỮA 2 MÁY TRONG MẠNG NỘI BỘ BẰNG WEBRTC

Ths. Lê Viết Long

I. MÔ TẢ YÊU CẦU

1. Mô hình kết nối

- Máy A và Máy B kết nối trong cùng mạng LAN.
- Sử dụng WebRTC để truyền video/audio trực tiếp (P2P).
- Không cần server trung gian để xử lý media.

2. Các thành phần chính

- HTML/CSS: Giao diện đơn giản với 2 khung video.
- JavaScript + WebRTC API: Xử lý kết nối, truyền video/audio.
- Signaling Server (Node.js/WebSocket): Truyền thông tin kết nối giữa 2 máy.

3. Quy trình hoạt động

- Máy A mở ứng dụng, cấp quyền truy cập camera & mic.
- Máy B mở ứng dụng, cũng cấp quyền tương tự.
- Signaling Server giúp 2 máy trao đổi thông tin ICE/STUN/TURN.
- Kết nối P2P thiết lập, 2 máy có thể gọi video trực tiếp.

4. Yêu cầu & Lưu ý

- Trình duyệt hỗ trợ WebRTC (Chrome, Firefox, Edge).
- Cả 2 máy phải trong cùng mạng LAN. (Sử dụng máy tính và điện thoại (android) kết nối nhau)
- Không cần TURN server do kết nối không đi qua NAT.

II. CÁC BƯỚC THỰC HIỆN

1. Cài đặt Node.js

- Node.js là một môi trường chạy JavaScript trên phía máy chủ. Nó được xây dựng dựa trên V8 JavaScript Engine (cùng công nghệ với trình duyệt Chrome) và cho phép chạy mã JavaScript bên ngoài trình duyệt.
- Trong WebRTC, cần có Signaling Server để các peer trao đổi thông tin kết nối.
- Node.js phù hợp để triển khai Signaling Server vì:
    - Xử lý WebSocket tốt → Giúp truyền tải dữ liệu nhanh giữa các client.
    - Hiệu suất cao với lượng kết nối lớn.
    - Dễ dàng cài đặt và triển khai.
- Thực hiện:
    - Truy cập https://nodejs.org
    - Chọn gói cài đặt phù hợp với Hệ điều hành (hướng dẫn này dùng Windows 64bit)
    - Lưu ý: Nhớ set path cho biến môi trường
- Sau khi cài đặt kiểm tra phiên bản. Mở command promt gõ lệnh sau:

- node –v *(dùng kiểm tra phiên bản node.js)*
- npm –v *(dùng để kiểm tra phiên bản của npm (Node Package Manager) – trình quản lý gói (quản lý việc cài đặt, các thư viện, các gói cập nhật) cho Node.js.)*

- Viết code thử nghiệm. Tạo file test.js với nội dung:
  console.log("Xin chào từ Node.js!");
- Chạy thử. Mở command promt gõ lệnh sau:
  Node test.js

2. Cài đặt OpenSSL

- Khi triển khai WebRTC trên một server riêng, bạn có thể cần OpenSSL để tạo chứng chỉ TLS/SSL tự ký nhằm hỗ trợ **HTTPS (dùng hỗ trợ truy cập camera và micro) và WSS (WebSocket Secure)**.
- Cài đặt:
  - Tải OpenSSL tại: https://slproweb.com/products/Win32OpenSSL.html
  - **Chạy file cài đặt** và chọn phiên bản phù hợp:
  - Thêm OpenSSL vào `PATH`
    - Mở **Start Menu** → Gõ **Environment Variables** → Chọn **Edit the system environment variables**
    - Chọn **Environment Variables...**
    - Tìm biến **Path**, nhấn **Edit...**
    - Thêm đường dẫn thư mục cài OpenSSL, ví dụ:
      `C:\Program Files\OpenSSL-Win64\bin`
    - Nhấn **OK** → Khởi động lại máy tính
  - Kiểm tra OpenSSL đã hoạt động chưa. Mở command promt gõ: *openssl version*. Nếu hiển thị thông tin version thì thành công

3. Cài đặt môi trường (sử dụng Command Promt)

- **Tạo thư mục dự án**
  mkdir webrtc-app
  cd webrtc-app
  mkdir public

- Khởi tạo thông tin dự án mặc định
  npm init –y
- Cài đặt gói express (HTTPS server) dùng để tạo API hoặc web server và ws (WebSocket server) Dùng để tạo WebSocket server hỗ trợ giao tiếp thời gian thực.
  npm install express ws https fs
- Tạo chứng chỉ SSL và copy vào thư muc certs
  openssl req -newkey rsa:2048 -nodes -keyout key.pem -x509 -days 365 -out cert.pem
  mkdir certs
  move key.pem certs
  move cert.pem certs

4. Tạo Signaling Server hỗ trợ HTTPS

- Là máy chủ trung gian giúp hai thiết bị (peer) trao đổi thông tin kết nối ban đầu trước khi thiết lập kết nối trực tiếp trong WebRTC.
- WebRTC hoạt động theo mô hình Peer-to-Peer (P2P), nhưng để hai peer có thể kết nối với nhau, chúng cần:
  - Trao đổi thông tin kết nối (SDP – Session Description Protocol).
  - Gửi ICE Candidates để tìm đường kết nối xuyên NAT/firewall.
  - Đồng bộ trạng thái (ai gọi ai, ai rời cuộc gọi, v.v.).
  - WebRTC không cung cấp sẵn Signaling Server, nên ta phải tự xây dựng bằng WebSocket, WebRTC DataChannels, MQTT hoặc HTTP. Ở hướng dẫn này sử dụng WebSocket với Node.js

- **Các bước thực hiện:**

1. Import các thư viện cần thiết

```
const fs = require('fs');
const https = require('https');
const express = require('express');
const WebSocket = require('ws');
const path = require('path');
```

- **`fs` (File System)**: Dùng để đọc tệp chứng chỉ SSL (`key.pem` và `cert.pem`).
- **`https`**: Tạo máy chủ HTTPS bảo mật.
- **`express`**: Framework giúp xử lý HTTP và phục vụ file tĩnh.
- **`WebSocket`**: Thư viện để tạo server WebSocket.
- **`path`**: Hỗ trợ xử lý đường dẫn file một cách linh hoạt.

---

2. Khởi tạo ứng dụng Express

```
const app = express();
```

- Khởi tạo một ứng dụng Express để xử lý HTTP.

---

3. Tải chứng chỉ SSL

```
let options;
try {
    options = {
        key: fs.readFileSync('./certs/key.pem'),
        cert: fs.readFileSync('./certs/cert.pem')
    };
} catch (error) {
    console.error('Lỗi tải chứng chỉ SSL:', error);
    process.exit(1);
}
```

- **`fs.readFileSync('./certs/key.pem')`** và **`fs.readFileSync('./certs/cert.pem')`**: Đọc file chứng chỉ SSL.

- Nếu có lỗi (ví dụ: file không tồn tại), in lỗi ra console và dừng chương trình với **`process.exit(1)`**.

---

4. Tạo server HTTPS

```
const server = https.createServer(options, app);
```

- **`https.createServer(options, app)`**: Tạo server HTTPS với chứng chỉ SSL và ứng dụng Express.
- **`app`** sẽ xử lý các request HTTP.

---

5. Phục vụ file tĩnh từ thư mục `public`

```
app.use(express.static(path.join(__dirname, 'public')));
```

- Dùng **Express** để phục vụ file tĩnh (HTML, CSS, JS) từ thư mục `public`.
- **`__dirname`**: Lấy đường dẫn thư mục hiện tại.
- **`path.join(__dirname, 'public')`**: Kết hợp đường dẫn chính xác trên mọi hệ điều hành.

---

6. Khởi tạo WebSocket Server

```
const wss = new WebSocket.Server({ server });
```

- **`new WebSocket.Server({ server })`**: Tạo WebSocket server chạy cùng HTTPS server.

---

7. Xử lý sự kiện khi client kết nối

```
wss.on('connection', ws => {
    console.log('Client mới kết nối');
    ws.send('Chào mừng bạn đến với WebSocket Server!');
```

- **`wss.on('connection', ws => {...})`**: Khi một client kết nối, sự kiện này được kích hoạt.
- **`console.log('Client mới kết nối')`**: In thông báo khi có client kết nối.
- **`ws.send('Chào mừng bạn đến với WebSocket Server!')`**: Gửi tin nhắn chào mừng đến client.

---

8. Xử lý tin nhắn từ client

```
    ws.on('message', message => {
        wss.clients.forEach(client => {
            if (client !== ws && client.readyState === WebSocket.OPEN) {
                client.send(message);
            }
        });
    });
```

- **`ws.on('message', message => {...})`**: Khi client gửi tin nhắn, sự kiện này được gọi.
- **`wss.clients.forEach(client => {...})`**: Duyệt qua tất cả các client đang kết nối.
- **`if (client !== ws && client.readyState === WebSocket.OPEN)`**:
  - Kiểm tra nếu client không phải là người gửi (**`client !== ws`**).
  - Kiểm tra nếu client vẫn đang kết nối (**`client.readyState === WebSocket.OPEN`**).
- **`client.send(message)`**: Gửi lại tin nhắn đến các client khác (trừ người gửi).

---

9. Xử lý khi client ngắt kết nối

```
    ws.on('close', () => {
        console.log('Client đã ngắt kết nối');
    });
});
```

- **`ws.on('close', () => {...})`**: Khi client đóng kết nối, sự kiện này được kích hoạt.
- **`console.log('Client đã ngắt kết nối')`**: In thông báo khi client rời khỏi.

---

10. Khởi động server

```
server.listen(3000, () => console.log('Server chạy trên
https://192.168.9.31:3000'));
```

- **`server.listen(3000, ...)`**: Lắng nghe kết nối trên cổng **3000**.
- **`console.log(...)`**: Hiển thị thông báo server đã chạy thành công.

5. Viết code xử lý cho client (index.html)

Phần 1: Cấu trúc cơ bản của trang HTML

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>WebRTC Video Call</title>
</head>
<body>
```

- `<!DOCTYPE html>`: Khai báo loại tài liệu là HTML5.
- `<html lang="vi">`: Xác định ngôn ngữ là tiếng Việt.
- `<meta charset="UTF-8">`: Sử dụng bảng mã UTF-8 để hiển thị ký tự tiếng Việt.
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">`: Tối ưu hóa trang web cho thiết bị di động.
- `<title>WebRTC Video Call</title>`: Tiêu đề của trang web.

Phần 2: Hiển thị video và nút điều khiển

```
    <video id="localVideo" autoplay playsinline></video>
    <video id="remoteVideo" autoplay playsinline></video>
    <button onclick="startCall()">Bắt đầu gọi</button>
    <button onclick="endCall()">Dừng cuộc gọi</button>
```

- `<video id="localVideo" autoplay playsinline></video>`: Hiển thị video từ camera của người dùng.
- `<video id="remoteVideo" autoplay playsinline></video>`: Hiển thị video của người đối diện.
- `<button onclick="startCall()">Bắt đầu gọi</button>`: Nút bấm để bắt đầu cuộc gọi.
- `<button onclick="endCall()">Dừng cuộc gọi</button>`: Nút bấm để dừng cuộc gọi.

Phần 3: JavaScript để thiết lập WebRTC

```
    <script>
        const localVideo = document.getElementById('localVideo');
        const remoteVideo = document.getElementById('remoteVideo');
        const peerConnection = new RTCPeerConnection();
        const ws = new WebSocket('wss://your_ip:3000');
```

- `const localVideo = document.getElementById('localVideo');`: Lấy phần tử video của người dùng.
- `const remoteVideo = document.getElementById('remoteVideo');`: Lấy phần tử video của người đối diện.
- `const peerConnection = new RTCPeerConnection();`: Tạo kết nối WebRTC giữa hai thiết bị.
- `const ws = new WebSocket('wss://your_ip:3000');`: Kết nối WebSocket tới server.

Phần 4: Lấy quyền truy cập camera và microphone

```
        navigator.mediaDevices.getUserMedia({ video: true, audio: true })
            .then(stream => {
                localVideo.srcObject = stream;
                stream.getTracks().forEach(track =>
peerConnection.addTrack(track, stream));
            });
```

- `navigator.mediaDevices.getUserMedia({ video: true, audio: true })`: Yêu cầu quyền truy cập vào camera và microphone.
- `.then(stream => { ... })`: Khi được cấp quyền, lấy video/audio và hiển thị lên `localVideo`.
- `stream.getTracks().forEach(track => peerConnection.addTrack(track, stream));`: Thêm luồng video/audio vào kết nối WebRTC.

Phần 5: Nhận luồng video từ người đối diện

```
        peerConnection.ontrack = event => {
            remoteVideo.srcObject = event.streams[0];
        };
```

- `peerConnection.ontrack = event => { ... }`: Khi nhận được video/audio từ người đối diện, gán vào `remoteVideo`.

Phần 6: Lắng nghe tin nhắn từ WebSocket

```
        ws.onmessage = async event => {
            try {
                let message;
                if (event.data instanceof Blob) {
                    const text = await event.data.text();  // Chuyển Blob
thành chuỗi
                    message = JSON.parse(text);
                } else {
                    message = JSON.parse(event.data);
                }
                handleMessage(message);
            } catch (error) {
                console.error("Lỗi xử lý WebSocket:", error, event.data);
            }
        };
```

- `ws.onmessage = async event => { ... }`: Khi nhận tin nhắn từ WebSocket, xử lý nội dung.
- Nếu dữ liệu nhận được là **Blob** (dữ liệu nhị phân), chuyển thành chuỗi JSON.
- Gọi `handleMessage(message);` để xử lý nội dung.

Phần 7: Xử lý tin nhắn WebSocket

```
        function handleMessage(message) {
            if (message.offer) {
                const acceptCall = confirm("Bạn có muốn chấp nhận cuộc gọi
không?");

                if (acceptCall) {
                    peerConnection.setRemoteDescription(new
RTCSessionDescription(message.offer))
                        .then(async () => {
                            const answer = await
peerConnection.createAnswer();
                            await peerConnection.setLocalDescription(answer);
                            ws.send(JSON.stringify({ answer }));
                        });
                } else {
                    console.log("Người dùng đã từ chối cuộc gọi.");
                }
            } else if (message.answer) {
                peerConnection.setRemoteDescription(new
RTCSessionDescription(message.answer));
            } else if (message.candidate) {
                peerConnection.addIceCandidate(new
RTCIceCandidate(message.candidate));
            }
        }
```

- Nếu **nhận được offer**, hỏi người dùng có muốn chấp nhận cuộc gọi không.
- Nếu chấp nhận, thiết lập `RemoteDescription`, tạo `answer`, gửi lại cho người gọi.
- Nếu **nhận được answer**, thiết lập `RemoteDescription` để kết nối hoàn tất.
- Nếu **nhận được ICE candidate**, thêm vào `peerConnection`.

Phần 8: Gửi ICE Candidate qua WebSocket

```
        peerConnection.onicecandidate = event => {
            if (event.candidate) {
                ws.send(JSON.stringify({ candidate: event.candidate }));
            }
        };
```

- Khi trình duyệt tìm thấy ICE Candidate, gửi nó qua WebSocket để kết nối ngang hàng.

Phần 9: Bắt đầu cuộc gọi

```
        async function startCall() {
            const offer = await peerConnection.createOffer();
            await peerConnection.setLocalDescription(offer);
            ws.send(JSON.stringify({ offer }));
        }
```

- Tạo **offer** cho cuộc gọi.

- Thiết lập **LocalDescription**.
- Gửi **offer** qua WebSocket để người nhận xử lý.

Phần 10: Dừng cuộc gọi

```
        function endCall() {
            localVideo.srcObject?.getTracks().forEach(track => track.stop());
            remoteVideo.srcObject?.getTracks().forEach(track =>
track.stop());

            peerConnection.close();
            ws.close();

            console.log("Cuộc gọi đã dừng.");
        }
```

- **Dừng luồng video** của cả hai bên.
- **Đóng kết nối WebRTC**.
- **Đóng WebSocket** (nếu cần).

5. Luyện tập tại lớp

Yêu cầu: Chỉnh sửa lại code trên để khi có nhiều client cùng kết nối server. Cho phép lựa chọn client để thực hiện gọi video.

6. Bài tập về nhà

Yêu cầu:

- Tìm hiểu kết nối thông qua Sever Turn (kết nối qua internet)
- Mở rộng: Cho phép tạo phòng / gọi video theo nhóm